

Welcome back! Mujhe khushi hai ki aapko Module 1 samajh aaya. Chaliye ab apne RAG system ko aur bhi smart banate hain.

Aaj hum padhenge **Module 2: MultiQuery Retriever**.

Module 2: MultiQuery Retriever

1. Why this concept exists

Humans hamesha perfect questions nahi puchte. Kabhi hum typos karte hain, kabhi humare words document ke words se alag hote hain (synonyms), ya kabhi hamari query bahut chhoti (vague) hoti hai. MultiQuery Retriever is problem ko solve karta hai by using the LLM to "think of multiple ways to ask the same question" (Query Expansion) taaki hume better search results milein.

2. Problem with traditional RAG (Why single query retrieval fails)

Basic RAG ek **Single Point of Failure** par chalta hai.

Agar user ne question pucha aur Vector Database mein exact ya close semantic match nahi mila, toh retrieval fail ho jayega.

Example: Document mein likha hai "Machine Learning models require high computational power."

User ne pucha: "Why is AI so expensive to run?"

Dono ka meaning same hai, par words bilkul alag hain. Single query search shayad is context ko miss kar de.

3. Real-world example

Maan lijiye aap ek E-commerce support bot bana rahe hain.

User types: "Mera order kab aayega?" (Very simple query).

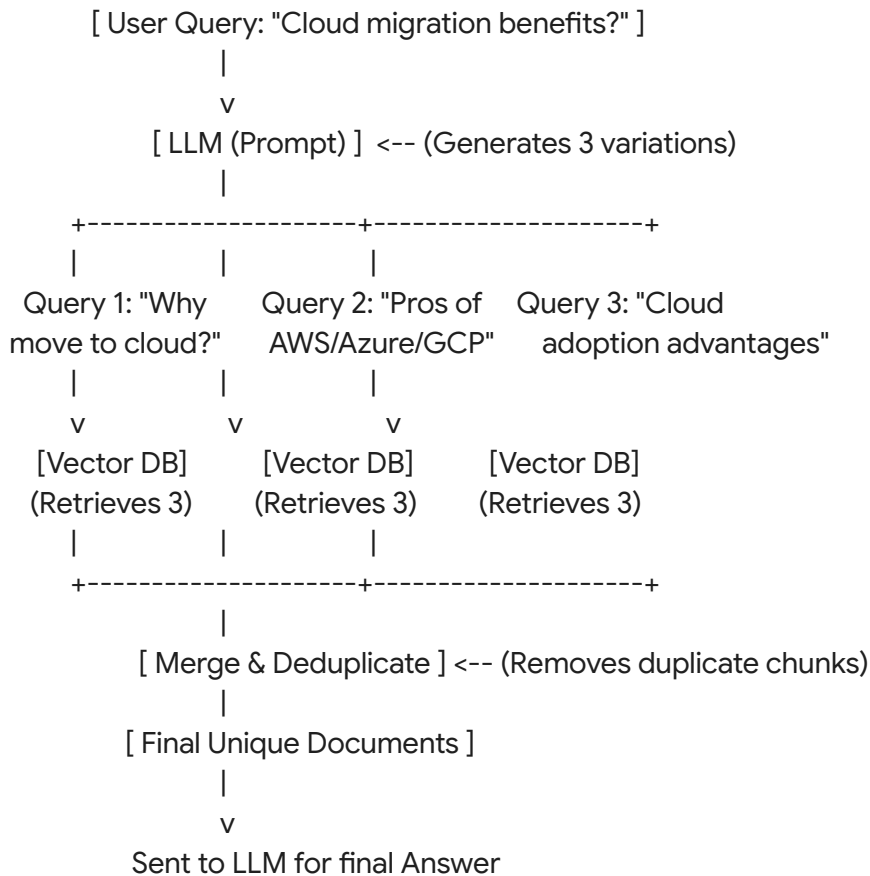
Agar hum isko directly search karein, toh ho sakta hai exact policy na mile.

Lekin agar hum isko **MultiQuery** mein daalein, toh LLM isko 3-4 professional queries mein expand kar dega:

1. "What is the standard delivery time for orders?"
2. "How to track shipping status?"
3. "Shipping and delivery policies."

Ab jab hum in teeno queries ko search karenge, toh 100% chance hai ki hume correct policy document mil jayega!

4. Visual explanation using diagrams made with text



5. Architecture flow

1. **Input:** User ek single natural language question deta hai.
2. **Expansion:** Ek LLM (jise hum query generator kehte hain) us question ko analyze karta hai aur uske 3 se 5 alag-alag variations (rewrites) generate karta hai.
3. **Multiple Retrievals:** Har ek generated query ke liye Vector Database mein alag se similarity search hoti hai.
4. **Union/Deduplication:** Saare retrieved documents ko ek jagah ikattha kiya jata hai, aur duplicates (jo chunks multiple queries mein common aaye) ko remove kiya jata hai.
5. **Generation:** Final unique documents context ke roop mein main LLM ko pass kiye jaate hain answer generate karne ke liye.

6. Step-by-step implementation

1. Apna standard Vector Store initialize karein.
2. Ek LLM instance banayein (preferably fast & cheap, like gpt-3.5-turbo ya llama3).
3. `MultiQueryRetriever.from_llm()` class use karein aur usme apna vector store aur LLM pass karein.
4. (Optional but recommended) logging enable karein taaki aap console mein dekh sakein ki LLM ne kya queries generate ki.
5. Retrieval chain banayein aur query run karein.

7. Complete Python code & LangChain implementation

```
import os
import logging
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains import create_retrieval_chain
from langchain_core.prompts import ChatPromptTemplate

# Setup API Key
os.environ["OPENAI_API_KEY"] = "sk-your-openai-api-key"

# 1. Logging setup so we can SEE the generated queries!
logging.basicConfig()
# MultiQueryRetriever uses the 'langchain.retrievers.multi_query' logger
logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)

# 2. Load Data (Let's use a web article for variety)
loader = WebBaseLoader("https://lilianweng.github.io/posts/2023-06-23-agent/")
docs = loader.load()

# 3. Split Text
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
splits = text_splitter.split_documents(docs)

# 4. Create Vector Store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)

# 5. Initialize the MultiQueryRetriever
# Hum GPT-3.5 ko bol rahe hain ki mere liye queries expand karo
llm_for_queries = ChatOpenAI(temperature=0)
```

```

retriever_from_llm = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm_for_queries
)

# 6. Create QA Chain
llm_for_answer = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
prompt = ChatPromptTemplate.from_template("""
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.

Context: {context}

Question: {input}
Answer:
""")

document_chain = create_stuff_documents_chain(llm_for_answer, prompt)
qa_chain = create_retrieval_chain(retriever_from_llm, document_chain)

# 7. Test the system!
user_question = "What is task decomposition in AI agents?"
print(f"User Question: {user_question}\n")

# When you run this, watch the console logs! You will see 3 new queries.
response = qa_chain.invoke({"input": user_question})

print("\n--- Final Answer ---")
print(response["answer"])

```

8. Production use case

Customer Support Portals: Jab users tickets raise karte hain, unki bhasha bahut casual aur unstructured hoti hai (e.g., "App crash ho gaya payment par"). Vector DB exact matches dhoondhne mein fail ho sakta hai. MultiQuery is short, casual query ko professional search terms mein expand kar deta hai ("Payment gateway application crash", "Troubleshooting payment errors in app"), jisse technical troubleshooting docs easily retrieve ho jate hain.

9. Interview questions

1. **Q: How does MultiQueryRetriever handle duplicate chunks returned by different queries?**

A: It uses Python sets or hash maps internally to perform a 'Union' operation based on the document's unique ID or content, ensuring the LLM doesn't receive redundant information.

2. **Q: Does MultiQueryRetriever increase latency? How can we fix it?**

A: Yes, it adds one LLM call upfront. We can reduce latency by using a faster, smaller LLM (like Llama-3-8B or Claude Haiku) purely for generating the queries, and multiple vector searches can be parallelized (LangChain handles this internally with async).

3. **Q: Can we customize the prompt used to generate the multiple queries?**

A: Yes! Aap PromptTemplate pass kar sakte hain class mein, jisme aap strict instructions de sakte hain (e.g., "Generate queries only in medical terminology").

10. Advantages

- **Higher Recall:** Kyunki hum alag-alag angles se search kar rahe hain, relevant chunk miss hone ka chance bahut kam ho jata hai.
- **Robustness to phrasing:** User chahe slang use kare ya synonyms, system handle kar lega.
- **Zero Shot Performance:** User ko perfect prompt likhne ki zaroorat nahi padti.

11. Limitations

- **Cost:** LLM ko do baar call karna padta hai (ek baar queries generate karne ke liye, ek baar answer ke liye).
- **Speed (Latency):** Extra LLM call aur multiple vector searches se response time thoda badh jata hai.
- **Context Window Full:** Agar 3 queries ne 4-4 documents laa diye, toh total 12 documents ho jayenge (minus duplicates). Ye token limit exceed kar sakta hai agar chunks bade hain.

12. Best practices

- **Use Cheaper LLMs for Expansion:** Queries generate karne ke liye GPT-4 ki zaroorat nahi hai. GPT-3.5 ya local models perfectly kaam karte hain. Cost aur speed dono bachenge.
- **Limit Top-K:** Apne base retriever mein search_kwargs={"k": 2} rakhein. Agar 3 queries run hongy, toh total 6 documents aayenge. Agar k=5 rakha, toh 15 documents aayenge, jo LLM ko overload kar sakte hain.
- **Custom Prompting:** Hamesha default prompt ke bharose mat raho. Domain-specific (legal/medical) app hai toh query generator ko prompt mein domain ka context do.

Assignments & Practical Exercises

- **Exercise 1:** Upar diye gaye code ko run karo aur logging ka output dekho. Phir, user_question ko change karke ek deliberate typo/spelling mistake (e.g., "Wat is tas decomp in AI?") dalo aur dekho kya LLM usko samajh kar sahi search queries banata hai!
- **Exercise 2:** LangChain docs padho aur MultiQueryRetriever ke andar ek Custom Prompt inject karo jo strictly Hindi aur English queries generate kare.

Common Mistakes Beginners Make

- *Forgetting to configure Logging:* Log level configure na karne par aapko pata hi nahi chalega ki system background mein kya queries fire kar raha hai, jisse debugging

impossible ho jati hai.

- *Using slow LLMs for query generation:* GPT-4 ko query expansion ke liye use karne se response aane mein 10-15 seconds lag sakte hain, jo user experience kharab karta hai.